Étudiant : Ilyas Fanan
Enseignant : Aounzou Youssef

Compte rendu TP 3

Monitoring et observabilité

Sujet	Supervision d’une application Flask
Outils	Docker Compose, Prometheus et Grafana

Résumé. Ce TP met en place une chaîne locale de supervision : une application Flask expose ses métriques, Prometheus les collecte et évalue une règle d’alerte, puis Grafana les présente dans un tableau de bord. Les vérifications réalisées confirment le démarrage des trois services, la collecte des compteurs et le déclenchement de l’alerte lors d’une panne simulée.

1. Objectif et architecture

Le monitoring indique si un service fonctionne et signale une anomalie. L’observabilité aide à en comprendre la cause en croisant plusieurs signaux. Dans ce TP, les métriques décrivent l’activité, les journaux donnent le contexte des événements et les traces, présentées comme troisième pilier, ne sont pas instrumentées.

L’application contient trois routes : / compte les visites normales, /erreur simule une réponse HTTP 500 et /metrics fournit les mesures au format Prometheus.

Composant	Rôle dans le TP	Accès
Flask	Application instrumentée et journaux INFO/ERROR	localhost:5000
Prometheus	Collecte toutes les 5 s et évalue les alertes	localhost:9090
Grafana	Visualise les mesures dans un tableau préconfiguré	localhost:3000
Docker Compose	Construit et relie les trois services localement	Réseau Compose

Les captures de la préparation, du démarrage et de la vérification de l’application sont regroupées dans l’annexe visuelle.

2. Mise en œuvre

2.1 Application Flask

L'application définit deux compteurs Prometheus. app_requetes_total est étiqueté par route afin de distinguer / et /erreur. app_erreurs_total augmente à chaque appel de la route d'erreur. Les appels normaux sont consignés au niveau INFO et les erreurs simulées au niveau ERROR.

2.2 Collecte et alerte Prometheus

Le job application interroge la cible app:5000 toutes les cinq secondes. La règle ApplicationArretee surveille up{job="application"} == 0. Elle passe en alerte si la cible reste indisponible pendant quinze secondes; son annotation indique que l'application ne répond plus.

2.3 Déploiement et tableau de bord

Le fichier Docker Compose décrit les services app, prometheus et grafana et publie respectivement les ports 5000, 9090 et 3000. La configuration a été contrôlée avec docker compose config, puis les conteneurs ont été construits et lancés en arrière-plan.

La source Prometheus de Grafana pointe vers http://prometheus:9090, adresse interne au réseau Compose. Le tableau Supervision application est chargé automatiquement et comporte trois panneaux : le débit de requêtes par route, le nombre d'erreurs et la disponibilité de l'application.

3. Vérifications et résultats

Les tests ont été exécutés après le démarrage de la stack. Trente requêtes ont été envoyées à /, puis dix requêtes à /erreur. Les valeurs ci-dessous ont été lues dans Prometheus et les journaux.

Vérification	Résultat observé	Conclusion
docker compose config	Configuration Compose acceptée	Services décrits correctement
up{job="application"}	1 lorsque Flask est démarré	Cible joignable
app_requetes_total	/ : 30 ; /erreur : 10	Compteur par route opérationnel
app_erreurs_total	10	Compteur d'erreurs opérationnel
Journaux de l'application	30 INFO et 10 ERROR	Événements lisibles dans les logs
Tableau Grafana	Source Prometheus et 3 panneaux chargés	Visualisation disponible
Arrêt puis redémarrage de Flask	DOWN puis UP ; alerte Firing puis résolue	Cycle d'alerte vérifié

4. Analyse

Les compteurs confirment que Prometheus distingue les deux routes et conserve le nombre d'erreurs. Le niveau de journal INFO ou ERROR facilite le repérage des appels ordinaires et des incidents.

Lors de l'arrêt de Flask, Prometheus détecte que la cible n'est plus joignable. Après le délai de quinze secondes défini dans la règle, l'alerte passe à Firing. Au redémarrage, la cible revient à UP et l'alerte se résout. Les résultats illustrent la différence entre détecter un problème et en chercher les indices dans les métriques et les journaux.

5. Conclusion

La stack locale remplit l'objectif du TP : Flask produit des mesures et des journaux, Prometheus collecte les mesures et surveille la disponibilité, et Grafana les rend visibles. Les fichiers de configuration permettent de relancer la supervision de façon reproductible. La prochaine étape pourrait être l'ajout d'Alertmanager, de Loki pour centraliser les journaux ou d'OpenTelemetry pour suivre les traces.

Annexe Captures d’écran

Les captures ci-dessous illustrent la préparation, la validation de la configuration, le lancement des services et la réponse HTTP de l’application.

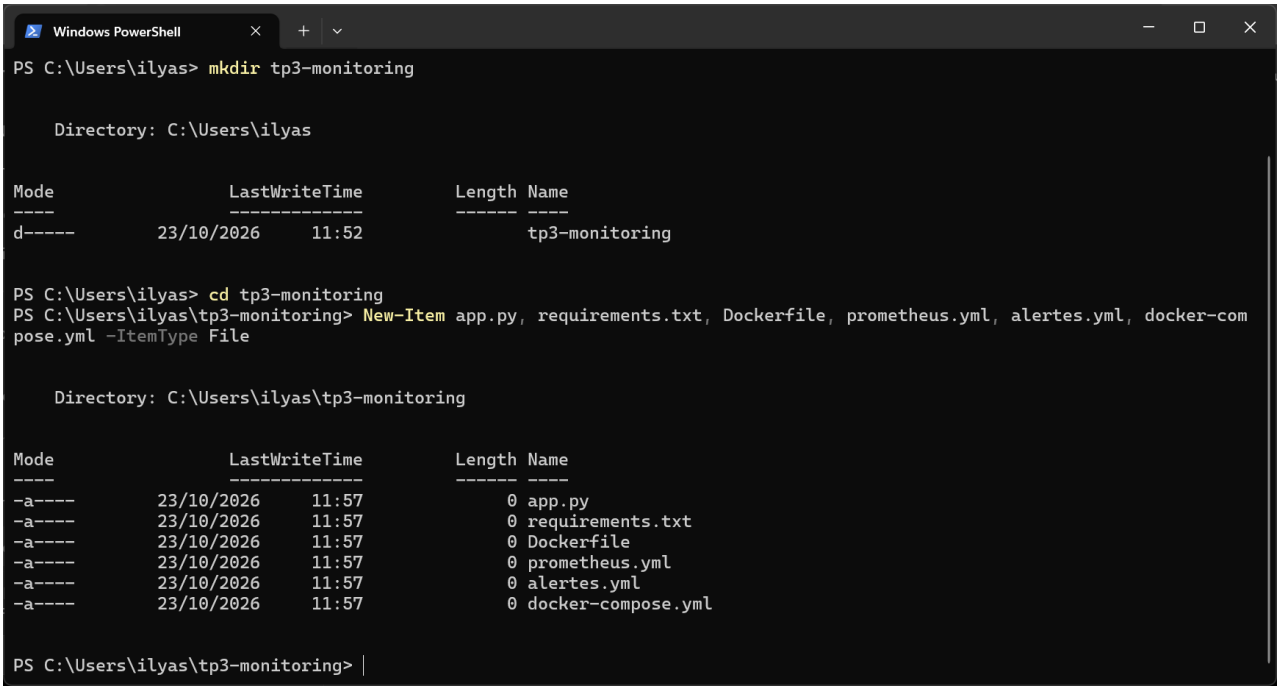


Figure 1. Préparation du dossier et création des six fichiers du TP dans PowerShell.

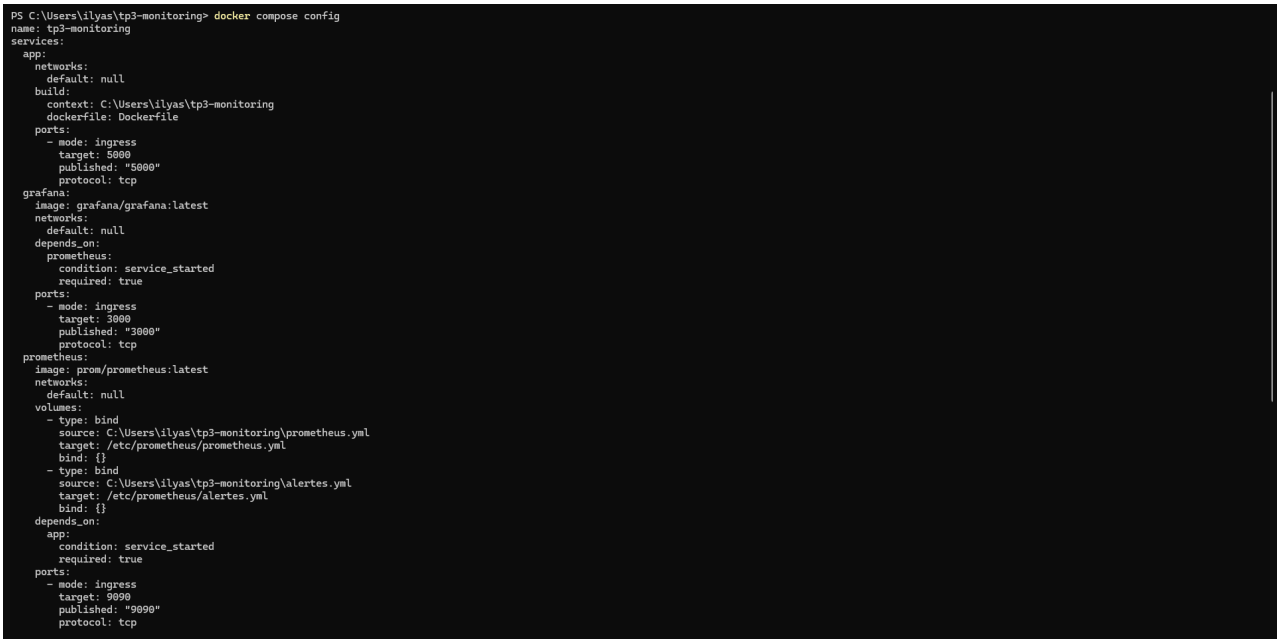


Figure 2. Vérification de la configuration Compose avec docker compose config.

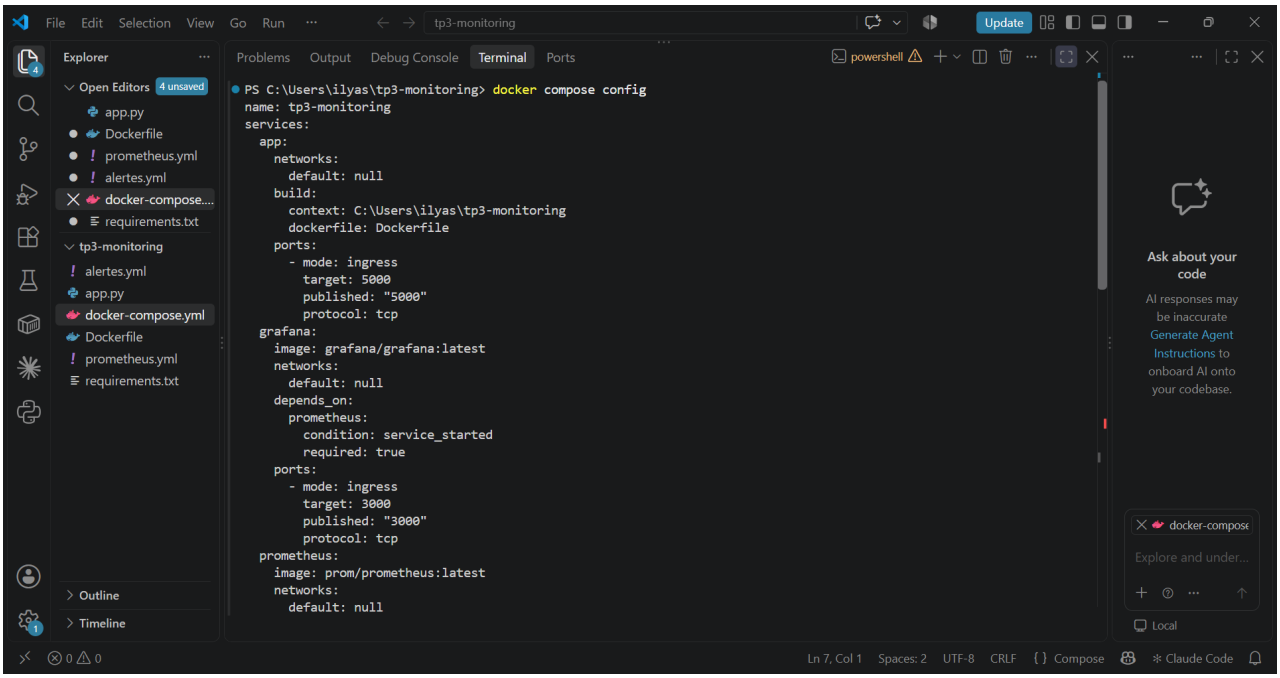


Figure 3. Configuration Compose affichée dans VS Code, première partie.

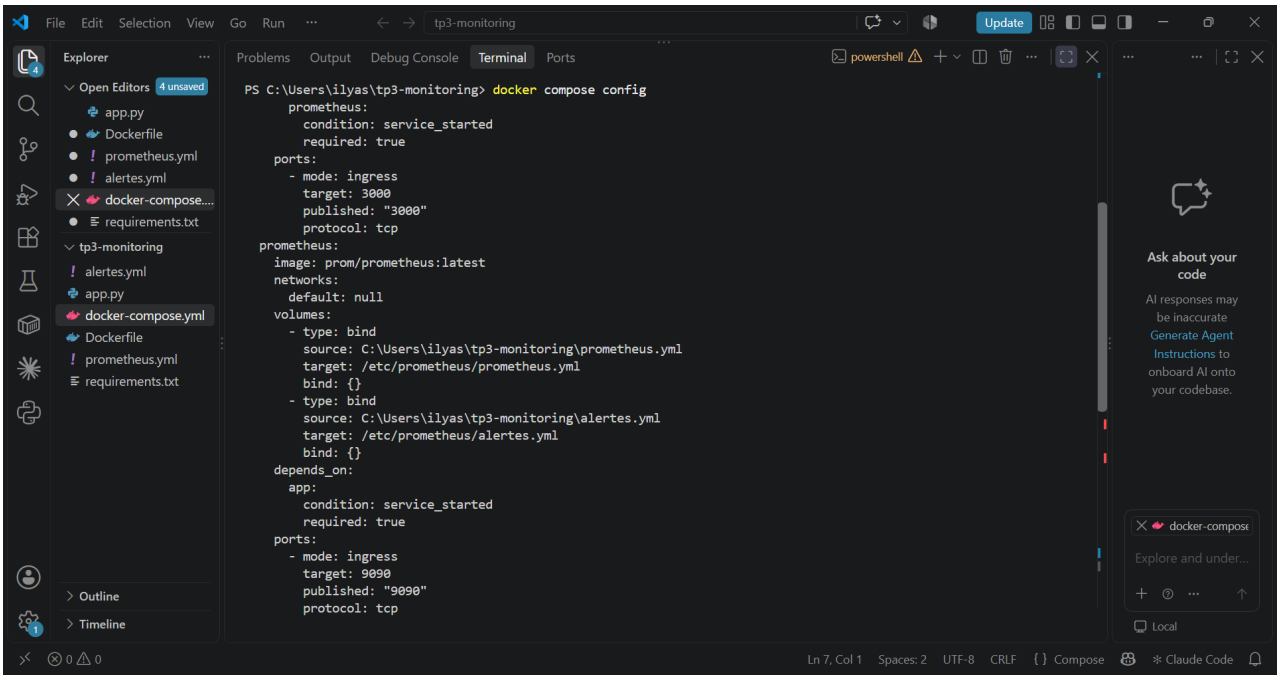


Figure 4. Suite de la configuration Compose affichée dans VS Code.

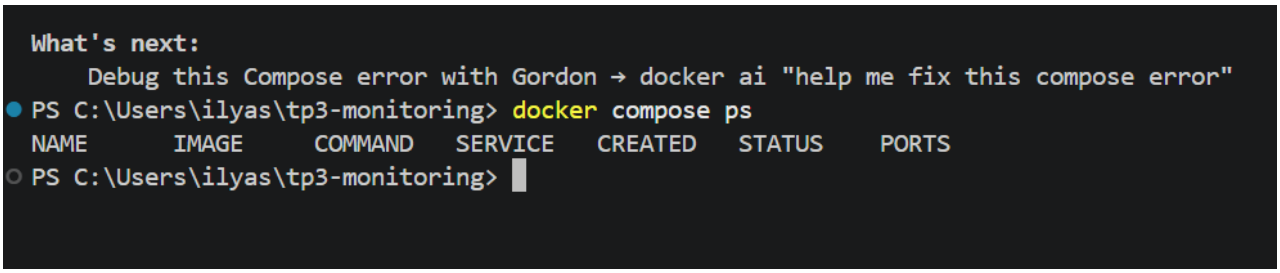


Figure 5. Avant le lancement, docker compose ps ne liste encore aucun service actif.

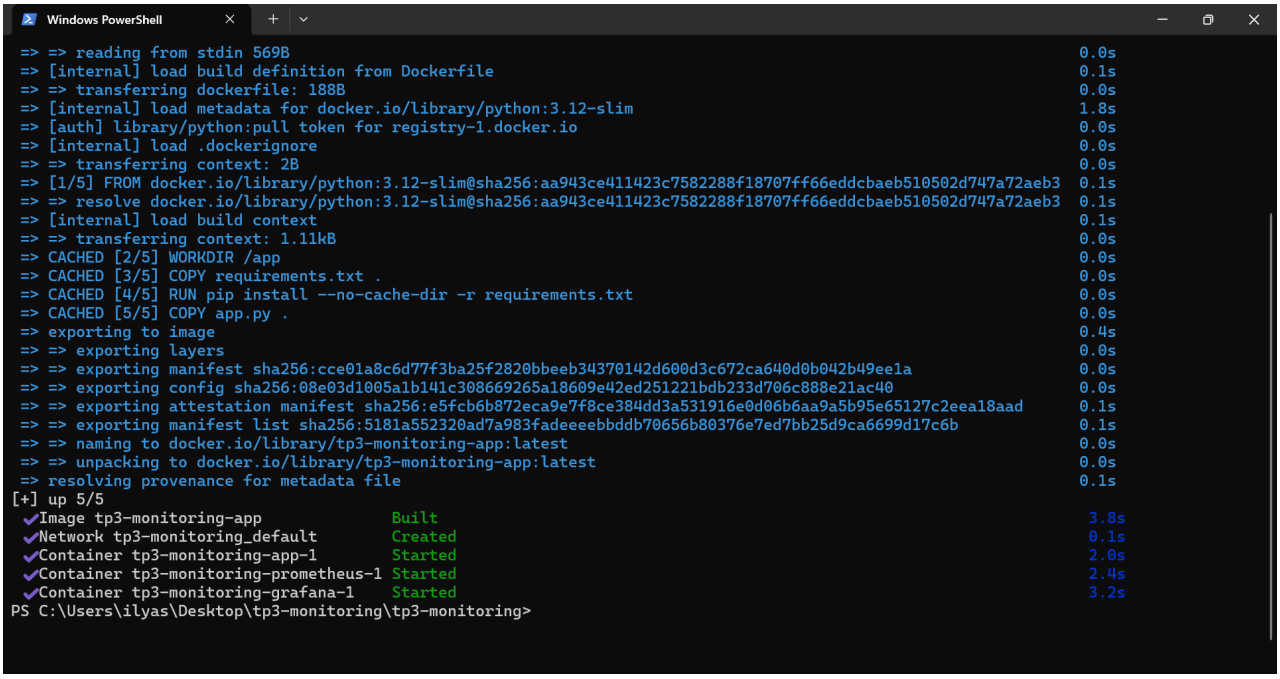


Figure 6. Construction de l'image Flask et démarrage des trois conteneurs.

```
PS C:\Users\ilyas\Desktop\tp3-monitoring\tp3-monitoring> docker compose ps
NAME                          IMAGE                      COMMAND                  SERVICE      CREATED          STATUS          PORTS
tp3-monitoring-app-1          tp3-monitoring-app         "python app.py"          app          2 minutes ago    Up 2 minutes    0.0.0.0:5
000->5000/tcp, [::]:5000->5000/tcp
tp3-monitoring-grafana-1      grafana/grafana:latest     "/run.sh"                grafana      2 minutes ago    Up 2 minutes    0.0.0.0:3
000->3000/tcp, [::]:3000->3000/tcp
tp3-monitoring-prometheus-1   prom/prometheus:latest     "/bin/prometheus --c…"   prometheus   2 minutes ago    Up 2 minutes    0.0.0.0:9
090->9090/tcp, [::]:9090->9090/tcp
PS C:\Users\ilyas\Desktop\tp3-monitoring\tp3-monitoring>
```

Figure 7. Après le démarrage, app, Prometheus et Grafana sont à l'état Up.



Figure 8. Réponse HTTP de l'application sur localhost:5000.